

Name:ImeldaMwaura

Reg Number : BIT/2024/39544

PROJECT DOCUMENTATION REPORT

PROJECT TITLE: STUDENT MANAGEMENT SYSTEM MOBILE APPLICATION

Course / Department: Computer Science and Information Technology

Institution: School of Computing and Informatics, Mount Kenya University

1. Introduction & Background

The Student Management System (SMS) is a cross-platform mobile application designed to streamline student access to critical campus services. In modern educational environments, students require immediate, reliable access to academic records, fee ledgers, and institutional notices. This project implements a modern mobile solution utilizing a decoupled data architecture and responsive design to deliver an optimal user experience across various device form factors.

2. Project Objectives

- **Cross-Platform Delivery:** Develop a single codebase capable of running seamlessly on both Android and iOS devices using the React Native framework.
- **Declarative Routing:** Implement file-based navigation via Expo Router to manage application states and user access controls cleanly.
- **Architecture Separation:** Isolate user interface (UI) presentation layers from the underlying data structures to mimic real-world production codebases.
- **Dynamic Content Consumption:** Integrate synchronous data simulation to handle live campus alerts and remote server data communication.

3. System Architecture & File Structure

The application utilizes **Expo Router**, a file-based routing engine built on top of React Navigation.

The directory structure is organized systematically to maintain scalability:

```
Plaintext
student-app/
├── app/
│   ├── _layout.tsx      # Root navigation stack configuration
│   ├── index.tsx        # Application bootstrapper and redirect logic
│   ├── login.tsx        # User authentication interface
│   ├── signup.tsx       # Account registration interface
│   ├── database.ts       # Isolated data model and mock data store
│   ├── dashboard.tsx    # Main user portal (Flexbox layout grid)
│   └── api-consumer.tsx  # Live network state and asynchronous data view
├── package.json          # Dependencies and entry point definitions
└── tsconfig.json         # TypeScript configuration environment
```

4. Weekly Implementation Breakdown

Week 1: Environment Setup & Core Navigation Infrastructure

- **Milestone:** Initialized a TypeScript-based Expo workspace.

- **Technical Output:** Configured package.json to leverage the expo-router/entry gateway. Developed the foundational _layout.tsx wrapper script implementing structural stack routing mechanisms with header inheritance disabled.

Week 2: User Authentication Interface & Multi-Screen Routing

- **Milestone:** Built the identity access screens and route forwarding rules.
- **Technical Output:** Created login.tsx and signup.tsx. Leveraged React hooks (useState) to manage local form parameters. Configured index.tsx as a silent automated pointer using Expo's <Redirect/> component to enforce authentication upon boot.

Week 3: Data Architecture Isolation

- **Milestone:** Decoupled structural entity models from presentation components.
- **Technical Output:** Established database.ts to implement strict TypeScript data profiling through the StudentProfile interface. Created an immutable mock object container to decouple data rendering from local view modules, preparing the system for live enterprise database hooks.

Week 4: The Responsive Dashboard Layout

- **Milestone:** Designed the primary system command hub.
- **Technical Output:** Implemented a full grid layout engine within dashboard.tsx utilizing **Flexbox** primitives (flexDirection: 'row', flexWrap: 'wrap'). Used layout ratios (width: '48%') to achieve multi-column fluid scaling across diverse viewport resolutions without layout breakage.

Week 5: Live API Simulation & Asynchronous Consumption

- **Milestone:** Integrated asynchronous request mapping to mimic production backends.
- **Technical Output:** Built api-consumer.tsx using the React lifecycle hook (useEffect) and simulated network timing loops. Implemented conditional interface logic using an <ActivityIndicator/> to gracefully render loading states during background processes before mapping incoming datasets to a <FlatList/> array renderer.

5. Technical Highlights & Design Choices

💡 Core Design Paradigm: Separation of Concerns (SoC)

By keeping user state definitions completely outside of the layout templates (via database.ts), the application achieves high maintainability. If the institution decides to switch data engines from mock files to a live remote PostgreSQL or Firebase database, the design code remains entirely untouched.

Flexbox Layout Optimization Matrix

The core UI elements adapt dynamically based on the following structural constraints:

- flex: 1: Ensures parent views consume the absolute maximum boundaries of physical device displays.
- flexWrap: 'wrap': Automatically forces overlapping interface nodes onto a fresh horizontal block when system typography or accessibility bounds shift.

- `elevation & shadowOpacity`: Adds platform-specific drop-shadow rendering for proper visual depth across Android and iOS hardware layers.

6. Conclusion & Future Scope

The Student Management System successfully fulfills all five development criteria milestones. The application features strict structural separation, reliable navigation stacks, fluid grid scaling, and modern asynchronous data rendering.

Future production upgrades will involve stripping out the simulated timing hooks in favor of native AXIOS/Fetch clients tied to live backend web servers and JSON Web Token (JWT) database middleware to ensure enterprise-grade user data security.

This report provides a strong academic framework for the project. Does your friend need any specific additions made to this template, such as use-case explanations or extra system details?